AI Agent 前置知识：从零开始理解智能体

> 这份文档是为完全零基础的读者准备的。如果你已经熟悉 AI Agent 的基本概念，可以直接翻阅主报告。

一、Agent 到底是什么？用一句话说清楚

Agent = 大模型 + 工具 + 记忆 + 自主行动。

传统的大模型（如 ChatGPT）是你问一句它答一句——你是那个主动方，它是被动的回答机。Agent 不一样，你给它一个目标（"帮我订一张下周去东京的机票"），它会自己规划步骤、调用工具查询航班、对比价格、确认行程，甚至主动问你几个关键问题，最终把事情办完。

打个比方：ChatGPT 是你的百科词典，Agent 是你的私人助理。

SOAR——历史上第一个 Agent 架构

AI Agent 不是 2023 年才有的概念。1983 年，卡内基梅隆大学的 John Laird 等人创建了 **SOAR（State, Operator And Result）**——历史上第一个可计算的通用智能体架构。SOAR 的核心思想是"在状态空间中搜索，不断选择操作、观察结果、再选下一步"，这个模式跟今天 Agent 的"思考→行动→观察"循环本质上是同构的。区别在于：SOAR 的操作（Operator）需要人手工编写规则，今天 Agent 的"操作"由大模型动态生成。

Symbolic AI（符号 AI）——为什么早期 Agent 没做成

1983–2010 年代的 Agent 基于 **Symbolic AI**——用人类手写的逻辑规则来表示知识。比如你写"如果温度低于 20 度，就开暖气"。这种方法的问题很明显：真实世界太复杂了，你没法把所有情况都写成规则。一个搬箱子机器人在模拟环境里完美运行，放到仓库里就会被阴影、灯光变化搞崩溃。Symbolic AI 的瓶颈不是规划算法，而是**感知和理解真实世界的能力**。直到 LLM 出现，Agent 才有了真正"理解世界"的引擎。

二、LLM 和 Agent 有什么区别?

维度	普通 LLM（大模型）	Agent（智能体）
主动性	被动回答，你问它答	主动规划，自主行动
工具使用	不能（只能输出文字）	能调用 API、查数据库、操作网页
记忆	会话内记忆（对话上下文）	有长期记忆，能记住之前的偏好和决定
任务形式	单轮问答	多步任务，需要自己拆解步骤
典型例子	你问"巴黎天气如何"，它回答	你让它"安排一次巴黎旅行"，它查天气、订酒店、做行程表

关键直觉：LLM 是大脑，Agent 是有了手脚和记忆的大脑。

三、Tool Calling（工具调用）——Agent 的"手"

Tool Calling（也叫 Function Calling）是 Agent 调用外部工具的能力。

它解决了什么问题?

普通 LLM 只能输出文字。如果用户问"今天北京的空气质量怎么样"，LLM 的知识截止日期可能是一年前，它没法知道今天的实时数据。有了 Tool Calling，LLM 可以输出一个结构化的请求：

```
调用函数：query_air_quality(city="北京")
```

系统收到这个请求后，真的去天气 API 查数据，把结果返回给 LLM，LLM 再用结果生成回答。

直观理解

```
用户："今天北京空气好吗？"
LLM（思考后）→ 调用 query_air_quality("北京")
          → 返回 "AQI: 85, PM2.5: 65"
          → LLM 整理成回答："今天北京空气质量良，AQI 85..."
```

2023 年 6 月，OpenAI 首次在 GPT-4 中引入 Function Calling，这是 Agent 历史上的一个关键节点。在此之前，让 LLM 调用工具需要写复杂的 prompt hack，非常不可靠。Function Calling 让工具调用变成模型的原生能力。

四、ReAct（推理+行动）——Agent 的"思考循环"

ReAct 是 "Reasoning + Acting" 的缩写，由普林斯顿大学和 Google 的研究者在 2022 年提出。

核心思想

Agent 不要一次性输出答案，而是**边思考边行动**，把推理过程和工具调用交错进行。

举个例子

假设 Agent 要回答"特斯拉 2024 年在中国卖了多少辆车？"

```
思考1：我需要知道特斯拉2024年的全球销量和中国占比
行动1：调用 search("特斯拉 2024 中国 销量")
结果1：特斯拉2024年中国销量约65.7万辆
思考2：还需要确认这个数据是否包含出口
行动2：调用 search("特斯拉 2024 中国 销量 包含出口吗")
结果2：不包含，仅限国内交付
思考3：数据可靠，可以回答
回答：特斯拉2024年在中国交付约65.7万辆（不含出口）
```

每一步都在"我想知道什么→去查→查到什么→下一步该做什么"之间循环。

为什么 ReAct 很重要?

没有 ReAct 的 Agent 就像蒙着眼睛做决策——推理了老半天，但得不到外部世界的验证。ReAct 让 Agent 能**用真实世界的信息纠正自己的推理偏差**。

一个关键背景：**大模型存在"幻觉"（Hallucination）问题**——模型会在不确定时编造看起来合理但实际错误的内容。没有外部验证的纯推理（CoT 模式）容易放大幻觉：模型在一本正经地推理一个错误的前提。ReAct 通过"推理→查证→再推理"的循环，用真实数据打断幻觉链条。

五、Planning（规划）——Agent 怎么决定先做什么

复杂任务不能一步到位，Agent 需要把大目标拆成小步骤。

Chain-of-Thought（思维链，CoT）

最简单的规划方式：让模型"一步一步思考"。Google 在 2022 年发现，在 prompt 里加一句"让我们一步一步来"，复杂推理的准确率能提升 30% 以上。

举个例子，问"超市里有 20 个苹果，卖掉了 7 个，又进了 3 箱每箱 12 个，现在有多少个？"

- 普通回答：直接输出 "49 个"（可能算错）
- CoT：先写推理过程 "20−7=13, 13+36=49" → 输出 "49 个"

Tree of Thoughts（思维树，ToT）

CoT 是一条直线。ToT 是一棵树：在每一步生成多个可能的思考方向，搜索最好的那条路。

比如解一个数学谜题，模型可以同时考虑 3 种不同的解法，评估每种方法有沒有前途，然后沿着最有希望的方向继续探索。

Graph of Thoughts（思维图，GoT）

GoT 是 2024 年的进一步扩展——从"树"变成"图"。树的分支不能合并，但图中的节点可以互相连接。这更适合需要"综合多个思路"的任务：Agent 同时探索 3 种解决方案，发现方案 A 和方案 B 中的某两个步骤可以合并，形成更好的方案 C。思维图允许 Agent **合并不同分支的洞察**，而不是死守一条路线。

Plan-and-Execute（先规划后执行）

ReAct 是边想边做。Plan-and-Execute 是**先想好再做**——先列出完整的步骤计划，然后按计划执行。适合步骤明确的流程化任务（比如"注册一个账号"）。

六、Memory（记忆）——Agent 怎么记住东西

Agent 的记忆分两个层次：

短期记忆（Context Window）

就是当前对话的上下文窗口。Claude 支持 200K token，Gemini 支持 1M+，相当于能一次性"记住"几百页文档。但 Agent 一旦开始新任务，旧对话就忘了。

长期记忆（Long-term Memory）

Agent 需要记住用户的偏好、历史决策、已完成的任务。实现方式通常是：

- **向量数据库（Vector DB）**：把信息转化为数学向量存起来，需要时搜索最相关的内容
- **RAG（检索增强生成）**：从外部知识库检索相关信息，拼到 prompt 里让模型参考
- **MemGPT / Letta**：专门的项目，让 Agent 像操作系统管内存一样管理自己的记忆——重要的放"内存"，不重要的归档到"磁盘"

一个直观的例子

你每周都让 Agent 订周五下午的会议室。没有长期记忆的话，它每次都要问"你一般订哪个会议室？几个人？需要投影吗？"。有了长期记忆，它记得你"每周五 14:00-16:00 订 3 楼小会议室，6 个人，不需要投影"。这就是记忆的价值。

七、Multi-Agent（多智能体协作）

有时候一个 Agent 不够用，需要多个 Agent 分工合作。

BDI 模型——多 Agent 的理论基础

BDI（Belief-Desire-Intention，信念-愿望-意图）模型是 1987 年由哲学家 Michael Bratman 提出的 Agent 理论框架。核心思想：一个理性的 Agent 不是每时每刻重新计算所有选择，而是"承诺"于当前计划，只在必要时才重新评估。

- **信念（Belief）**：Agent 对世界的认知（"数据库里有 100 条记录"）
- **愿望（Desire）**：Agent 想达成的状态（"把数据库备份到云端"）
- **意图（Intention）**：Agent 当前正在执行的计划（"步骤 1：连接云端 → 步骤 2：传输数据"）

这个模型至今仍是多 Agent 系统设计的理论基础。

涌现（Emergence）——多 Agent 的"1+1>2"

多 Agent 系统最迷人的特性是**涌现**——多个简单的 Agent 各自执行自己的任务，却在整体上表现出意想不到的复杂行为。2023 年 Stanford 的 "Generative Agents" 论文把 25 个 Agent 放进一个模拟小镇，它们自主社交、组织派对、传播新闻，没有人写脚本规定它们"应该举办派对"——这个行为是自发涌现出来的。这就是多 Agent 比单 Agent 更有想象空间的原因。

典型的例子

写一份市场分析报告：

- **研究员 Agent**：负责收集数据、查资料
- **分析师 Agent**：负责解读数据、提炼洞察
- **写手 Agent**：负责把分析结果写成报告
- **编辑 Agent**：负责检查格式、校对细节

四个 Agent 像一个小团队一样协同工作。

多 Agent 框架的代表

- **AutoGen**（微软）：Agent 之间通过对话协作，像几个人在聊天群里讨论问题
- **CrewAI**：给每个 Agent 分配角色（Role）和任务（Task），像管理一个项目团队
- **MetaGPT**：模拟整个软件公司的角色——产品经理、架构师、工程师、测试员各司其职

八、MCP（Model Context Protocol）——Agent 的"USB-C"

MCP 是 Anthropic 在 2024 年底推出的**开放标准协议**，用来统一 Agent 和外部工具之间的连接方式。

它解决了什么问题?

在 MCP 出现之前，每个 Agent 框架都要自己写代码来对接不同的工具。对接一个 GitHub API 要写一段代码，对接一个 Slack 又要写另一段。就像每个设备都需要自己的充电线。

MCP 定义了**一套通用的协议**：工具提供方（MCP Server）按标准格式暴露功能，Agent（MCP Client）按标准格式调用。从此新工具只需实现一个 MCP Server，所有兼容 MCP 的 Agent 都能直接用。

MCP vs A2A

- **MCP**（Model Context Protocol）：Agent 连接工具的协议（垂直方向）
- **A2A**（Agent-to-Agent，Google 2025 年推出）：Agent 之间互相通信的协议（水平方向）

OpenClaw：个人 Agent 操作系统

2025 年底诞生的开源项目，到 2026 年 4 月 GitHub Star 数突破 30 万，超越 Linux 和 React 成为 GitHub 软件类星标历史第一。理念很直接：在你自己的电脑上跑一个 Agent 系统，让它自主管理文件、浏览网页、操作各种应用——不需要经过 App 的 UI。"用户不想用更多 App，他们想要一个能替自己操作所有 App 的 Agent"。之后 Hermes Agent（2026 年 3 月，Nous Research）以"持久记忆+自我进化"为差异化迅速崛起，10 万+ Stars，形成 OpenClaw 追求"广度"、Hermes Agent 追求"深度"的双足鼎立格局。

打个比方：MCP 是 USB 线（设备连电脑），A2A 是网线（电脑连电脑）。两者互补，不冲突。

九、Prompt Injection——Agent 的安全漏洞

这是 Agent 面临的最大安全威胁。

什么是 Prompt Injection?

通俗说：**有人通过输入内容黑进你的 Agent。**

普通 LLM 被注入最多是输出奇怪的回答。Agent 被注入，会导致它**真的去执行危险操作**——删除数据库、发送恶意邮件、修改密码。

真实案例

2026 年 4 月，Hacker News 上有一篇爆帖："我们 的 AI Agent 删除了生产数据库"。Agent 收到了一条包含恶意指令的输入，导致它调用了一个删除数据库的命令。这是 Agent 安全问题的标志性事件。

防护手段

- **权限最小化**：Agent 默认只读，需要特殊授权才能执行写操作
- **沙箱隔离**：Agent 在隔离环境中运行，无法触及核心系统
- **人工确认**：危险操作必须有人点击确认
- **内容过滤**：对输入内容做安全检查

十、Agent 基准测试——怎么衡量 Agent 好不好

基准测试	测试什么	简单理解
SWE-Bench	能不能修 GitHub 上的代码 bug	给 Agent 一个真实 Issue，看它能不能修好
GAIA	通用 AI 助手能力（多步推理+工具使用）	给一个复杂问题，看 Agent 能不能自己找资料解决
AgentBench	8 类真实任务（操作系统、网页、数据库等）	全面考核 Agent 在不同环境下的表现
WebArena	在真实网页上完成任务的能力	让 Agent 像人一样操作网页
OSWorld	操作桌面操作系统	让 Agent 用鼠标键盘操作电脑

目前人类在 SWE-Bench 上的解决率约 85%，最强的 Agent（Claude + Agent 框架）约 49%，说明 Agent 还有很大提升空间。

十一、Agent 框架 vs Agent 产品——别搞混了

Agent 框架（开发者工具）

给程序员用的，用来构建 Agent 应用的工具包：

框架	谁开发的	特点
LangChain	LangChain Inc	最流行的 Agent 框架，生态最丰富
LangGraph	LangChain Inc	LangChain 的 Agent 编排引擎，用"状态图"控制 Agent 流程
Dify	LangGenius	可视化界面，不用写代码也能做 Agent
AutoGPT	个人项目	2023 年引爆 Agent 概念的项目，183k stars，自主任务分解
AutoGen	微软	多 Agent 对话框架
CrewAI	CrewAI Inc	角色扮演式多 Agent 协作
OpenAI Agents SDK	OpenAI	OpenAI 官方 Agent 开发包，核心机制是 Handoff（Agent 间移交控制权）
Coze / 扣子	字节跳动	国内最流行的低代码 Agent 平台，月活领先

Agent 产品（终端用户工具）

给普通人用的成品：

产品	公司	能做什么
Manus	Monica	通用自主 Agent，帮你完成复杂任务
OpenAI Operator	OpenAI	操作浏览器完成网购、订票等
Claude Computer Use	Anthropic	操控电脑桌面
Cursor	Anysphere	AI 编程 IDE，帮你写代码
Devin	Cognition AI	AI 软件工程师，端到端修复 bug

十二、Token（词元）——理解 Agent 成本的关键

Token 是什么?

Token 是大模型处理文本的最小单位。简单理解：**模型不读"字"，它读"词元"。**

- "你好世界" = 4 个 token（你/好/世/界）
- "Hello world" = 2 个 token（Hello/ world）
- 一篇 1000 字的中文文章大约 = 800–1500 个 token

为什么 Token 对 Agent 很重要?

Agent 跑一次任务的 token 消耗是普通聊天的几十倍。原因：

1. **多轮推理**：Agent 每一步推理都要消耗 token（"想了什么→查到了什么→下一步该做什么"）
2. **工具返回结果**：调用 API 返回的数据要拼回 prompt 给模型看
3. **长上下文**：记忆和历史记录让上下文越来越长

按目前主流 API 价格（每百万 token 约 $3–15），一个复杂 Agent 任务可能消耗 50 万–200 万 token，单次成本 $1.5–$30。这就是 Agent 商业化最大的瓶颈——**用得起 vs 用得值**之间的平衡。

十三、Agent-as-a-Service——Agent 怎么赚钱

目前 Agent 的商业模式还在探索中，主要有几种：

1. **订阅制**：按月收费（Manus $39/月，Cursor $20/月）
2. **按用量计费**：根据 Agent 的工作量收费（GitHub Copilot 新模式）
3. **平台抽成**：提供 Agent 搭建平台，抽成插件/API 收入
4. **企业定制**：为大客户做私有化部署

核心矛盾：Agent 跑一次任务消耗的 token 是普通聊天的几十倍，成本太高是商业化最大的瓶颈。

十四、一张图总结

```
用户说："帮我计划一次东京旅行"
                      ↓
┌──────────────────────────────────────┐
│            Agent（智能体）              │
│                                      │
│  1. 推理（ReAct）：需要查天气、机票、酒店   │
│  2. 规划：先查日期 → 再查价格 → 再下单     │
│  3. 调用工具（Tool Calling）：            │
│     - search_flights("东京", 日期)       │
│     - search_hotels("东京", 日期)        │
│  4. 记忆：记住用户偏好（靠窗座位、禁烟房）   │
│  5. 输出：完整的行程方案                  │
└──────────────────────────────────────┘
                      ↓
用户收到："已为您订好 5月10日 国航CA925...
           酒店在新宿华盛顿，禁烟双床房..."
```